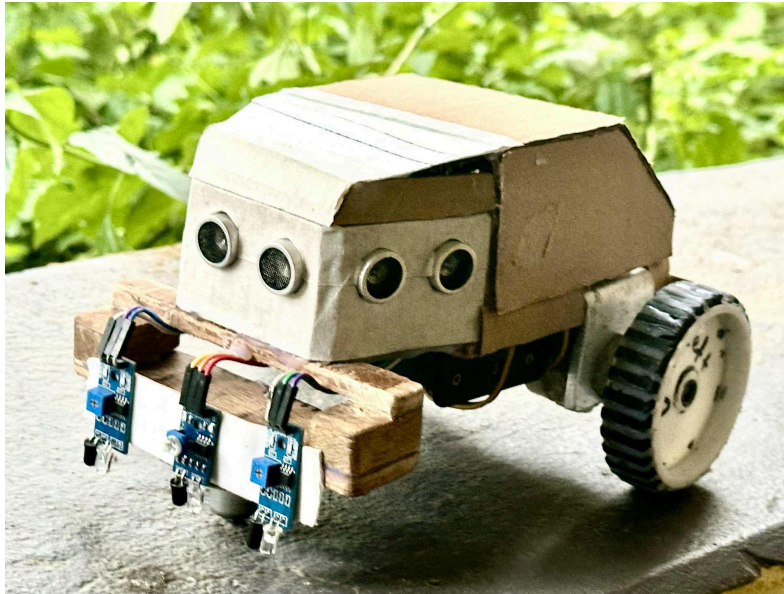


Robotics Project



PROJECT

MEGATRON KP2S2

A line following
and wall following bot

TEAM – B5

A group of five Enthusiasts

IOE, PASCHIMANCHAL CAMPUS

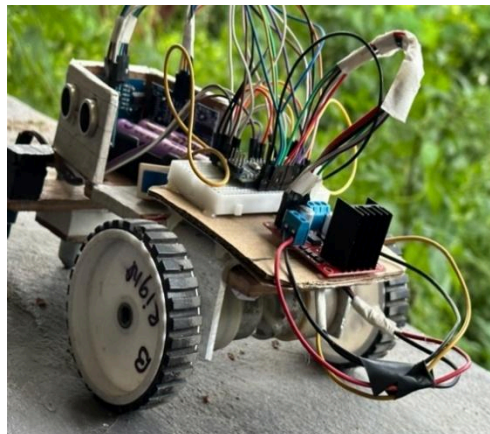
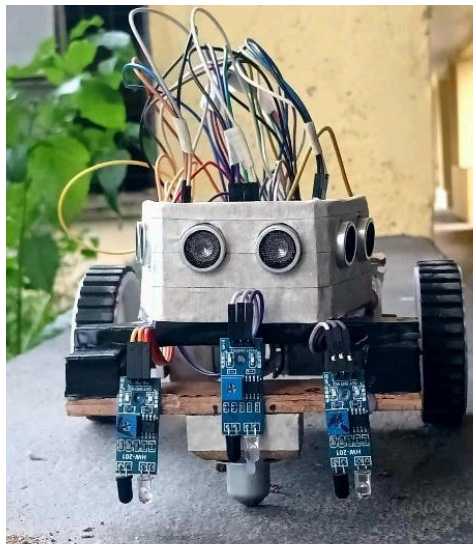
Table of contents

Introduction.....	1
Background.....	2
Component and material.....	3
Circuit Diagram.....	4
Software/codes.....	3
Methodology.....	4
Testing and Results.....	5
Conclusion.....	6
Team member.....	7
Reference.....	8

Introduction

This project presents an autonomous robot named **Megatron KP2S2**, designed to perform both **line-following** and **obstacle avoidance** using basic electronics and sensor integration. The primary goal is to develop a smart bot capable of navigating its environment by following a white path and intelligently reacting to obstacles or junctions along the way.

The robot is built using an **Arduino Nano**, **three IR sensors** for line detection, and **three ultrasonic sensors** for obstacle and wall sensing. It is powered by a **3-cell Li-ion battery** and uses **two DC motors** controlled by an **L298N motor driver**. This type of robot has real-world applications in areas like **warehouse automation**, **delivery systems**, and **educational robotics**.



Background

Line-following robots use **IR sensors** to detect surface colors based on infrared light reflection. In this robot, the IR sensors give **high analog readings when over the black surface** and **low readings when over the white line**. By analyzing the pattern of readings from the left, middle, and right IR sensors, the Arduino Nano decides how to steer the robot to keep it aligned with the white line.

If **all three IR sensors detect high readings**, it means the robot is no longer on the white line. This triggers a mode switch from line-following to **obstacle avoidance** using ultrasonic sensors. The ultrasonic sensors measure the distance to nearby walls or objects by sending out sound waves and measuring the return time of the echo. Based on this data, the robot makes decisions such as turning or moving forward to navigate its environment.

The **Arduino Nano** acts as the brain, reading sensor data and controlling the motors through an **L298N motor driver**. This combination of **IR-based path tracking** and **ultrasonic obstacle avoidance** forms a simple but intelligent robotic system suitable for indoor navigation and learning applications.

Components and materials

ELECTRONIC AND ELECTRICAL COMPONENTS

1. Arduino nano
2. Ultrasonic sensors(Three)
3. Motor driver
4. IR sensor(Three)
5. Jumper wires
6. Motor (Two)
7. Battery
8. Battery holder
9. Breadboard

PHYSICAL COMPONENTS

1. Wooden board
2. Wheels(two)
3. Cardboard
4. Caster Wheel

Electrical and Electronic components

Arduino Nano

Arduino is a microcontroller electronic platform that allows easy interaction between hardwares. There are various types of arduino including arduino uno, arduino nano, arduino due, arduino micro, arduino zero and so on. In this project, we have used Arduino nano. **Arduino nano** is a small, open-source microcontroller board that is breadboard-friendly. It is 45mm long, 18 mm wide and weighs 7 grams. It has male input output headers and a mini-B USB connector. It is programmed using the Arduino IDE which is an integrated development environment that is common to all Arduino boards. It can be powered by a 9V battery or a type-B mini USB cable. It has high-performance and low-power capabilities which can be used to develop security-based applications, robotics and many more.

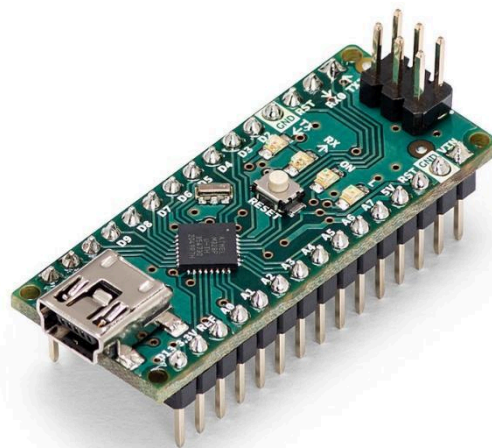


Figure: Arduino Nano

Motor Driver

A **motor driver** is an electronic device that **controls the speed and direction of an electric motor**. It serves as an interface between a microcontroller or other control system and the motor itself. They are essential for ensuring the safety of motors and preventing overcurrent and burnout. They are used in many applications including robotics, automation, electric vehicles and home appliances. It allows the motor to **rotate in different directions** and **control speed** by adjusting the **pulse with modulation signal**. There are various types of motor driver available such as stepper motor driver, DC motor driver, servo motor driver and so on. Some of the popular motor driver IC includes L298N, L293D, A4988 and many more. In this project we have used **L205N IC motor driver**.

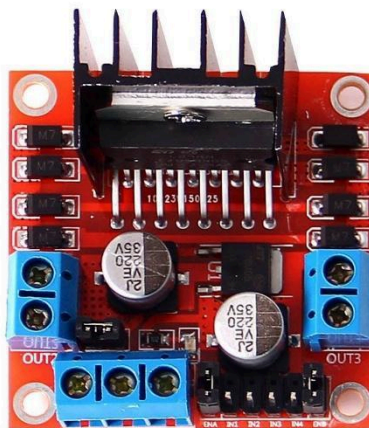


Figure: Motor Driver

Ultrasonic Sensor

Ultrasonic sensor is an electronic device that uses sound waves to measure distance and detect the presence of objects without making contact. They consist of a transmitter and a receiver.

They function by **emitting high-frequency sound waves** called **ultrasonic pulses** which is the job of a **transmitter** and further how long it takes for the waves to bounce back from an object.

The sensor then calculates the distance between the object by detecting the reflected wave by the receiver and the sensor using the speed of sound. They have a wide variety of applications such as anti-collision detection, people detection, contouring, trash collection vehicles, pallet detection, drink filling machines and many more.



Figure: An Ultrasonic Sensor

IR Sensor

IR Sensor is an electronic device that detects objects and measures distance based on infrared light without any physical contact. It consists of an **IR LED (transmitter)** and a **photodiode or phototransistor (receiver)**. The transmitter emits infrared light, and when an object is present in front of the sensor, the light reflects back and is detected by the receiver. The amount of reflected light helps the sensor determine the presence and distance of the object. IR sensors are commonly used in various applications such as **line-following robots, obstacle detection, proximity sensing, remote controls, flame detection, motion tracking, and automated light systems**. They are reliable, fast, and energy-efficient, making them ideal for both industrial and consumer electronics, **especially in line-following robots where they detect black and white lines based on reflected infrared light**.

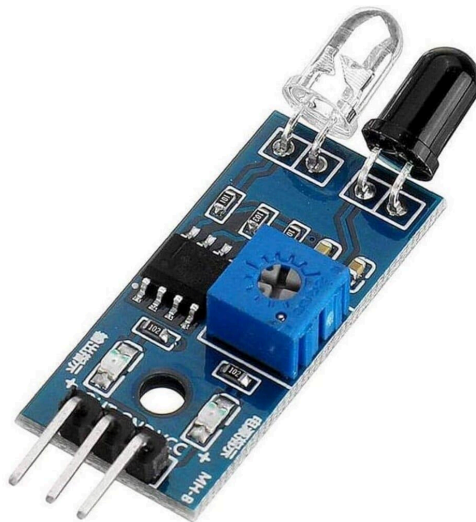


Figure: Infrared sensor

DC Gear Motor

DC Gear Motor (12V) is a motor integrated with a gearbox that reduces speed and increases torque, making it ideal for applications like robotics, automation, and DIY projects. It runs on 12 volts and is commonly used to drive wheels, arms, or other moving parts where controlled motion and power are needed.



Figure: DC Gear Motor

Lithium-Ion Battery (3.7V)

Three rechargeable lithium-ion batteries are used in series to provide a total of about **11.1V** for powering the entire system.



Figure: Li-ion Battery

Battery Holder

A 3-cell battery holder securely connects the lithium-ion batteries in series and provide reliable power output to the circuit.



Figure: 3-cell Battery holder

Breadboard

Breadboard is a solderless prototyping board used to build and test electronic circuits easily. It allows quick and temporary connections between components like sensors, microcontrollers, and motor drivers without the need for permanent wiring. In this bot, the breadboard is used to connect and organize all components neatly during testing and development.

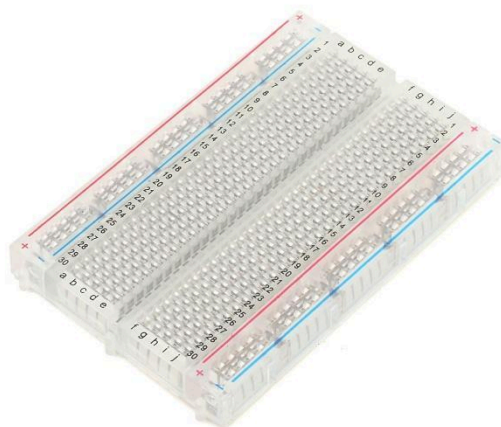


Figure: Breadboard

Jumper Wires

Jumper Wires are flexible electrical wires used to make quick and temporary connections between components on a breadboard or other circuit boards. They come in male-to-male, male-to-female, and female-to-female types. In this bot, jumper wires are used to connect the IR sensors, ultrasonic sensors, motor driver, and microcontroller efficiently without soldering.



Figure: Jumper Wires

Chassis design

We have utilized a wooden ply board for the chassis design, a lightweight and sturdy material. The chassis serves as the foundational component of the robot, linking all the motors and sensors together. The quality of the chassis directly impacts the performance of the robot. An exceptional chassis is not solely determined by its design but also by its mechanical attributes and dimensions. While our guideline suggests a maximum dimension of 20 cm x 20 cm for the bot, this is not obligatory; hence, our bot's dimension includes 20 cm x 9 cm. Opting for a smaller chassis enables more precise movements compared to a larger one.

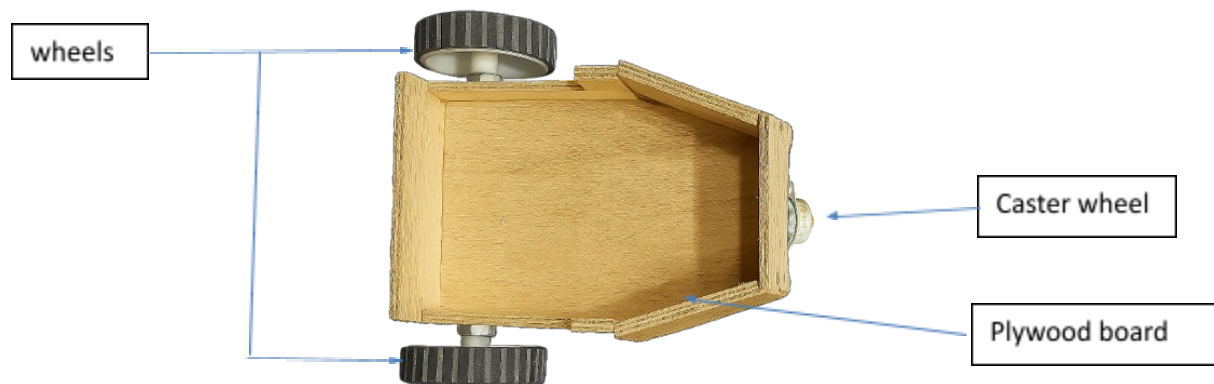
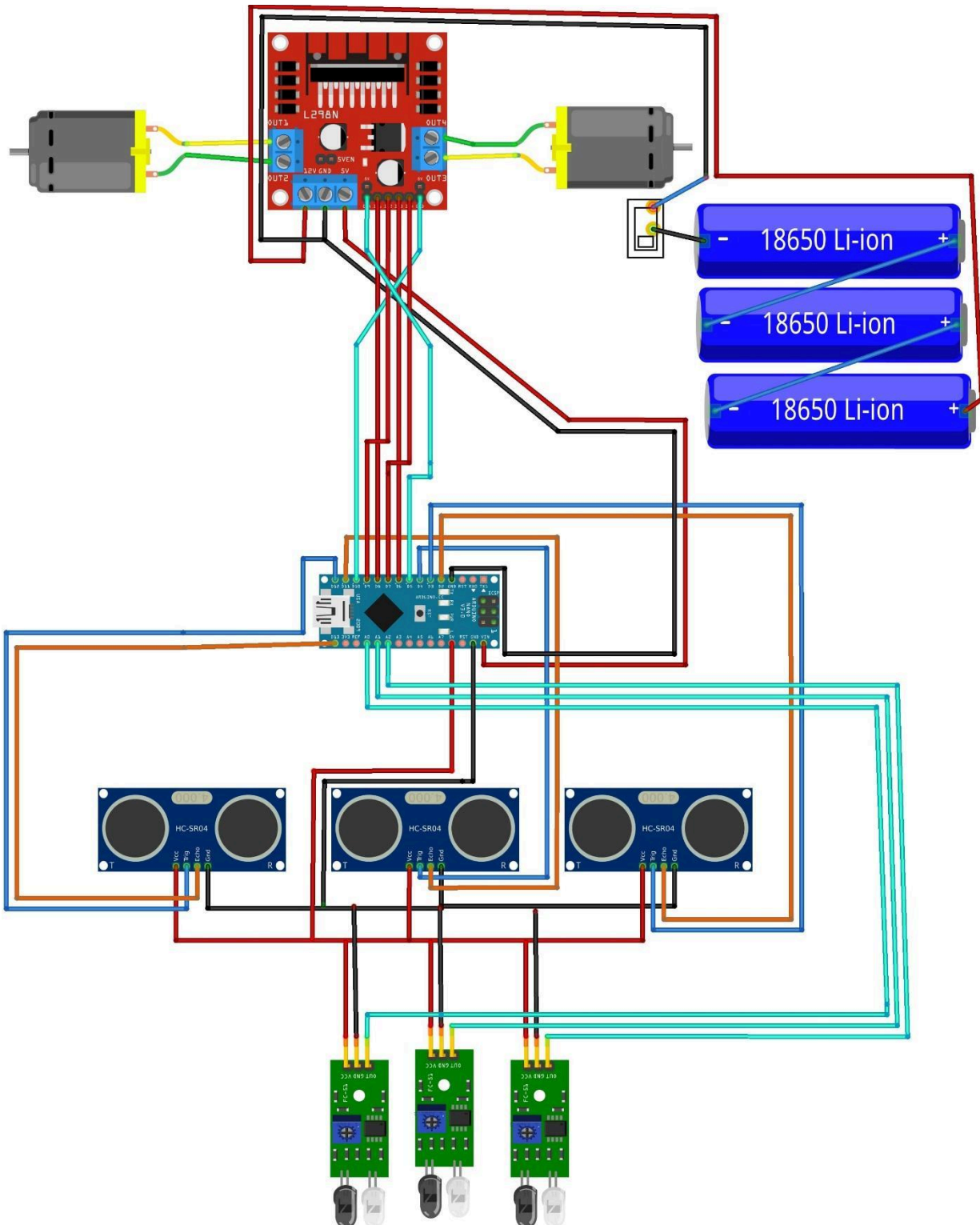


Figure: Chasis Design

COMPLETE CIRCUIT DIAGRAM



Arduino programming

Arduino programming is carried out using the Arduino Integrated Development Environment (IDE). It uses a simplified version of the C++ programming language, with built-in functions and methods designed for easy hardware interaction. Programming requires **an Arduino board**, which the code is uploaded to, allowing the board to perform specific tasks based on the instructions. Arduino makes it easy to work with components like **sensors, motors, LEDs, and displays**. It's beginner-friendly, simple to learn, and fully open-source, making it accessible for students, hobbyists, and professionals. It also provides a wide range of libraries and built-in functions that help simplify complex tasks.



Figure: Arduino IDE

Codes:

```
// put your main code here, to run repeatedly:

// === IR Sensor Pins ===

const int trigFront = 4;
const int echoFront = 11;

const int trigLeft  = 3;
const int echoLeft  = 2;

const int trigRight = 12;
const int echoRight = 13;

const int irLeftPin  = A2; // Left IR
const int irMidPin   = A1; // Middle IR
const int irRightPin = A0; // Right IR

// === Motor Driver Pins ===

const int IN1 = 8; // Right Motor Forward
const int IN2 = 9; // Right Motor Backward
const int IN3 = 6; // Left Motor Forward
const int IN4 = 7; // Left Motor Backward
const int ENA = 5; // PWM for Right Motor
const int ENB = 10; // PWM for Left Motor

// === Speed Settings ===
const int baseSpeed = 200; // Base speed
const int turnSpeed = 0; // Speed while turning
const int baseSpeed2 = 255;
```

```

const int sharpTurnThreshold = 10;

void setup() {
    // Motor pins as output
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);
    pinMode(ENA, OUTPUT);
    pinMode(ENB, OUTPUT);

    // IR sensors as input
    pinMode(irLeftPin, INPUT);
    pinMode(irMidPin, INPUT);
    pinMode(irRightPin, INPUT);

    pinMode(trigFront, OUTPUT); pinMode(echoFront, INPUT);
    pinMode(trigLeft, OUTPUT);  pinMode(echoLeft, INPUT);
    pinMode(trigRight, OUTPUT); pinMode(echoRight, INPUT);

    Serial.begin(9600);
}

void loop() {
    int leftIR  = analogRead(irLeftPin);
    int midIR   = analogRead(irMidPin);
    int rightIR = analogRead(irRightPin);

    // You might need to calibrate this threshold
    int threshold = 500;

    bool leftDetected = (leftIR < threshold); // Black = low
value

```



```

bool midDetected    = (midIR    < threshold);
bool rightDetected = (rightIR < threshold);

// Logic
if (midDetected && !leftDetected && !rightDetected) {
    // Perfectly on line
    moveForward(baseSpeed,baseSpeed);
} else if (leftDetected && !midDetected && !rightDetected) {
    // Line is to the left
    // moveForward(baseSpeed, turnSpeed);
    sharp(1);
} else if (rightDetected && !midDetected && !leftDetected) {
    // Line is to the right
    // moveForward(turnSpeed, baseSpeed);
    sharp(2);
} else if (leftDetected && midDetected && !rightDetected) {
    // Slight left
    sharp(1);
    // moveForward(baseSpeed, turnSpeed);
} else if (rightDetected && midDetected && !leftDetected) {
    // Slight right
    sharp(2);
    //moveForward(turnSpeed, baseSpeed);
} else if (leftDetected && rightDetected && !midDetected) {
    // Line break or junction
    //moveForward(baseSpeed, baseSpeed);
    moveForward(turnSpeed, baseSpeed);
} else if (!leftDetected && !midDetected && !rightDetected) {
    // Lost line
    ultra();
    //``stopMotors();
} else {

```

```

        // Default: go forward
        moveForward(baseSpeed, baseSpeed);
    }

    delay(10);
}

// === Motor Control Functions ===
void moveForward(int leftSpeed, int rightSpeed) {
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
    analogWrite(ENA, rightSpeed);
    analogWrite(ENB, leftSpeed);
}

void stopMotors() {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
}

void sharp(int error){
    if(error==1){
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        digitalWrite(IN3, LOW);
        digitalWrite(IN4, HIGH);
        analogWrite(ENA, baseSpeed-20);
        analogWrite(ENB, baseSpeed-20);
    }
}

```

```

    }
    else if(error==2){
        digitalWrite(IN1, LOW);
        digitalWrite(IN2, HIGH);
        digitalWrite(IN3, HIGH);
        digitalWrite(IN4, LOW);
        analogWrite(ENA, baseSpeed-20);
        analogWrite(ENB, baseSpeed-20);
    }

}

void ultra(){
int frontDist = getDistance(trigFront, echoFront);
    int leftDist = getDistance(trigLeft, echoLeft);
    int rightDist = getDistance(trigRight, echoRight);
    int total = leftDist + rightDist;
    if ((frontDist < sharpTurnThreshold) ) {
        if(total < 30){
            if (leftDist > 15 ) {
                slightRight();
            }
            else if (rightDist > 12 ) {
                slightLeft();
            }
            else {
                moveForward2();
            }
        }
    }
    else {
        if (leftDist > rightDist) {
            sharpLeft();
        } else {

```

```

        sharpRight();
    }
}
}
else {
    // Wall-following behavior
    if (leftDist > 15 ) {
        slightRight();
    }
    else if (rightDist > 12 ) {
        slightLeft();
    }
    else {
        moveForward2();
    }
}

}

int getDistance(int trigPin, int echoPin) {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    long duration = pulseIn(echoPin, HIGH, 30000); // timeout
after 30ms
    int distance = duration * 0.034 / 2;
    return constrain(distance, 0, 200); // safety cap
}

void moveForward2() {
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);

```

```

    analogWrite(ENA, baseSpeed2);
    analogWrite(ENB, baseSpeed2);
}

void slightLeft() {
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);    // Right
forward
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);    // Left
forward
    analogWrite(ENA, baseSpeed2);
    analogWrite(ENB, turnSpeed+50);    // Left slower
}

void slightRight() {
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);    // Right
forward
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);    // Left
forward
    analogWrite(ENA, turnSpeed+50);    // Right slower
    analogWrite(ENB, baseSpeed2);
}

void sharpRight() {
    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);    // Right
forward
    digitalWrite(IN3, HIGH);    digitalWrite(IN4, LOW);
    // Left forward
    analogWrite(ENA, baseSpeed2);
    analogWrite(ENB, baseSpeed2);
}

void sharpLeft() {

```

```
    digitalWrite(IN1, HIGH);  digitalWrite(IN2, LOW);  // Right
backward
    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
// Left backward
    analogWrite(ENA, baseSpeed2);
    analogWrite(ENB, baseSpeed2);
}
```

Function Explanation

Main Control Functions

void loop()

This is the main loop that keeps running repeatedly. It:

- Reads values from IR sensors.
- Decides movement based on sensor readings.
- Calls movement or obstacle avoidance functions.

Decision Helper Functions

void moveForward(int leftSpeed, int rightSpeed)

Moves the robot forward with specified speeds for the left and right motors.

- IN1, IN3 set HIGH to rotate both motors forward.
- Speed is controlled by analogWrite on ENA (right) and ENB (left).

void stopMotors()

Stops the robot:

- All motor direction pins set LOW.
- PWM speed is set to 0.

void sharp(int error)

Makes the robot take a sharp turn in place:

- `error == 1`: Left sharp turn → Left motor backward, right motor forward.
- `error == 2`: Right sharp turn → Right motor backward, left motor forward.

Used when:

- The bot needs to make every turn.

void ultra()

Handles obstacle avoidance using ultrasonic sensors:

- Reads front, left, and right distance.
- If an obstacle is close in front:
 - If there's enough space on the left or right, makes a slight turn.
 - If both sides are blocked, makes a sharp turn.
- If no obstacle in front:
 - Follows wall-like logic using slight turns or moves forward.

int getDistance(int trigPin, int echoPin)

Calculates distance using an ultrasonic sensor:

- Sends a short pulse on `trigPin`.
- Measures time until echo received on `echoPin`.
- Converts duration to distance in cm.
- `constrain()` limits result between 0 and 200 cm.

Motion Sub-functions (Used by ultra)

void moveForward2()

Same as `moveForward()` but uses `baseSpeed2` (full speed) instead of `baseSpeed`.

void slightLeft()

Makes the bot turn slightly left while moving forward:

- Both motors forward.
- Left motor runs slower → curves left.

void slightRight()

Makes the bot turn slightly right while moving forward:

- Both motors forward.
- Right motor runs slower → curves right.

void sharpRight()

Sharp right turn:

- Right motor goes backward, left motor goes forward.
- Causes the robot to rotate to the right in place.

void sharpLeft()

Sharp left turn:

- Left motor goes backward, right motor goes forward.
- Causes the robot to rotate left in place.

Methodology

There were some methods. They are listed below:

- Both the IR sensor and the Ultrasonic sensor should be placed in a way that they don't block each other or give wrong readings.
- To avoid conflict, only use the relevant sensors based on the mode.
- There must be a switch mode, like a switch, auto switch based on conditions.
- Make sure your power supply can handle both sensors and motors running together.
- Handle sensor timing delays carefully so readings don't interfere with each other.
- Keep code clean and modular.
- Make sure your chassis has enough space for all sensors.
- Sensor and Motor should be properly calibrated.

Testing and Results

The bot successfully executed both line-following and wall-following tasks. The IR sensors demonstrated reliable detection of black lines under various lighting conditions, ensuring stable line tracking. The wall-following algorithm effectively maintained a consistent distance from the walls. However, minor adjustments in sensor sensitivity and control logic are recommended to enhance performance, particularly around tighter corners. The mode is switched automatically according to the conditions detected by the sensors.

Conclusion

In this project, we successfully designed and implemented a dual-mode robot capable of both line following and wall following. By integrating relevant sensors—IR sensors for line detection and ultrasonic sensors for distance measurement—we enabled the robot to switch intelligently between modes based on environmental cues.

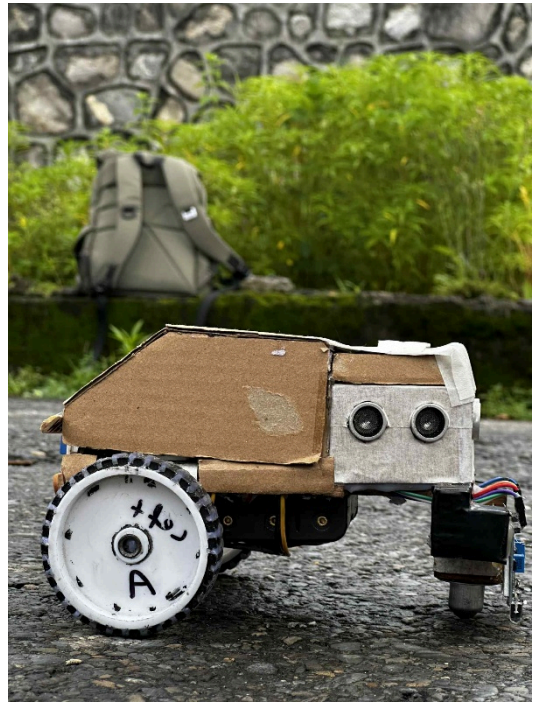
The modular approach in both hardware and software allowed for easier development, testing, and maintenance. Through careful calibration of sensors and tuning of motor controls, the robot demonstrated reliable performance in following lines accurately and maintaining a consistent distance from walls.

Overall, this project illustrates how combining multiple behaviors in a single robotic platform can be effectively managed with thoughtful calibration and control strategies. It provides a strong foundation for future work in autonomous navigation and multimodal robotics.

FINAL LOOK OF THE PROJECT



Back View



Side View



Front View

Team Members

Pawan Tripathi

Electronics Engineering

Kritan Bastola

Electronics Engineering

Sumina Chaudhary

Mechanical Engineering

Swarnim Ranabhat

Computer engineering

Prakat Sharma

Computer Engineering

Mentor

Nikhil Gautam

Mechanical engineering

Reference

- <https://www.arduino.cc/en/Tutorial>
- <https://www.geeksforgeeks.org/arduino-coding-basics>
- Tinkercard.com



THANK YOU!!